

Phase 1 Task List — Identity, Directory, Inventory

Sep 26, 2026 · @BigPhu

Phase 1 is delivered in six milestones, in order: platform foundation, identity, activity, directory, inventory core, then import/export. Each milestone ends with something that runs and can be demonstrated; later milestones depend on the earlier ones, but tasks inside a milestone can mostly be done in parallel. Tick a box when the task meets the definition of done at the bottom; structure and conventions follow the Project Structure Guidelines doc.

Milestone 0: Project setup and platform foundation

Outcome: `make dev` starts Postgres, `cmd/migrate` applies migrations, `cmd/api` serves a health check, and `cmd/worker` runs River with the event worker registered.

Repository and tooling

- ☐ Create the repository, `go.mod`, and the folder layout from the guidelines (`cmd/`, `api/`, `migrations/`, `internal/platform/`, empty module folders).
- ☐ Add `docker-compose.yml` with PostgreSQL for local development.
- ☐ Add Makefile targets: `dev`, `generate`, `migrate`, `migrate-new`, `run-api`, `run-worker`, `test`, `lint`.
- ☐ Declare tool dependencies in `go.mod`: `oapi-codegen v2`, `sqlc` (or pin the `sqlc` binary version in the Makefile), `goose`.
- ☐ Add `.golangci.yml` with the standard linters plus `depguard` rules for layer and module boundaries.
- ☐ Set up CI: `build`, `make lint`, `make test`, fail if `make generate` changes any committed file.

Platform packages

- ☐ `config`: typed `Config` from environment variables (database URL, JWT secret, app origin, storage settings); fail on missing values.
- ☐ `logger`: slog JSON setup and `logger.With(ctx)` adding request ID and actor ID.

- ☐ `database`: `Open(ctx, url)` for pgxpool and `WithTx(ctx, pool, fn)`.
- ☐ `errs`: `Error` type with kind, code, message; constructors for `Invalid`, `NotFound`, `Conflict`, `Unauthenticated`, `Forbidden`.
- ☐ `web`: `JSON`, `Error` (kind → status, 500 logged with request ID), `RequestError` for malformed requests, `Handle` for plain handlers.
- ☐ `api/common.yaml` with `Error`, pagination and ID schemas; generate `platform/web/apicommon`.
- ☐ `server`: `http.Server` with timeouts and `Run(ctx)` with graceful shutdown.
- ☐ `middleware`: `RequestLogger` (method, path, status, duration, request ID, actor).
- ☐ `jwt`: `Manager` with `Issue(accountID, permissions)` and `Verify(token)`; HS256, issuer check, expiry required; unit tests for expired, wrong algorithm and tampered tokens.
- ☐ `auth`: `Actor`, context helpers, `Require(ctx, perm)`, `SystemActor`, `bearer Middleware` with a list of public operations.
- ☐ `storage`: `Store` interface with local-disk implementation; S3-compatible implementation can wait until deployment.
- ☐ `jobs`: insert-only River client for the API and worker client for `cmd/worker`, with queue names and retry defaults.

Events (outbox with River)

- ☐ Migration for `platform.events` (ID, type, aggregate type and ID, actor ID, occurred at, JSON payload).
- ☐ `events.Event` envelope and `Registry` (event type → named subscribers, plus `OnAll`).
- ☐ `Outbox.Append(ctx, tx, events...)`: insert event rows and one unique `HandleEvent` job per subscriber with `InsertManyTx`.
- ☐ `HandleEventWorker`: load the event, restore the actor from `ActorID`, call the subscriber, let River retry on error.
- ☐ Integration test: a rolled-back transaction leaves no event and no job; a committed one leaves both.

Binaries

- ☐ `migrations/embed.go` and `cmd/migrate` applying goose migrations and River migrations.

- ☐ `cmd/api` : config, pool, registry, outbox, chi router with middleware, `/healthz` , graceful shutdown.
- ☐ `cmd/worker` : config, pool, registry, River worker client with `HandleEventWorker` , graceful shutdown.
- ☐ Test helper: start Postgres with testcontainers-go and apply all migrations once per test package.

Milestone 1: Identity

Outcome: an admin can log in, receive a JWT, call `GET /me` , and manage accounts and roles; every other module can check permissions.

Database

- ☐ Migration for `identity.accounts` (ID, email unique, name, password hash, member ID, active, version, timestamps).
- ☐ Migration for `identity.roles` , `identity.role_permissions` , `identity.account_roles` .
- ☐ Seed the system roles Admin, Asset Manager, Viewer and Member with their permissions.
- ☐ sqlc queries: get account by email/ID, list account permissions, insert/update account, role CRUD, assign roles.

Domain and service

- ☐ `domain` : `Account` with `CanLogin` , `Role` , errors, repository interfaces, identity permission constants.
- ☐ `service.Login` : lowercase email, bcrypt compare, same error for unknown email and wrong password, issue JWT with permissions.
- ☐ `service.Me` : return the current account from the actor.
- ☐ `service` account management: `CreateAccount` , `DisableAccount` , `AssignRoles` , `SetPassword` , `ChangePassword` (minimum 12 characters).
- ☐ `service` role management: `ListRoles` , `CreateRole` , `UpdateRolePermissions` ; system roles cannot be deleted.
- ☐ Account and role changes append events (`identity.account_created` , `identity.roles_assigned` , ...).

- ☐ `contract`: `AccountReader` (name and email by ID) for showing "changed by" in other modules.
- ☐ Bootstrap: create the first admin from `ADMIN_EMAIL / ADMIN_PASSWORD` on first start if no accounts exist.

HTTP

- ☐ `openapi.yaml`: `POST /auth/login`, `GET /me`, `/accounts`, `/accounts/{accountID}`, `/accounts/{accountID}/roles`, `/roles`, `/roles/{roleID}`, `GET /permissions`.
- ☐ Generate the strict server and implement all operations; mark `POST /auth/login` public in `auth.Middleware`.
- ☐ Generate the TypeScript client for the frontend.

Tests

- ☐ Domain and service unit tests, including disabled accounts and permission-denied cases.
- ☐ Repository integration tests.
- ☐ Handler tests: login success, wrong password (401), missing token (401), missing permission (403).

Milestone 2: Activity (audit trail)

Outcome: every identity change shows up as a history entry within seconds, with who did it and when; later modules get history for free by appending events.

- ☐ Migration for `activity.entries` (ID, event ID unique, entity type, entity ID, action, actor ID, occurred at, JSON data with before/after values).
- ☐ `domain`: `Entry` and `EntryRepository`.
- ☐ `service.Record(ctx, event)`: turn any event into an entry; idempotent via `ON CONFLICT (event_id) DO NOTHING`.
- ☐ `service.History(ctx, entityType, entityID, page)`: newest first, with actor names from identity's `AccountReader` contract.
- ☐ `subscriber/record.go`: registered with `registry.OnAll("activity.record", ...)` in `module.go`.
- ☐ `openapi.yaml`: `GET /history/{entityType}/{entityID}`; permission checks per entity type (for example `inventory.asset.read` for assets).

- ☐ Wire the module into `cmd/api` (handler, registry) and `cmd/worker` (subscriber).
- ☐ Tests: a retried event does not create a duplicate entry; entries appear for identity events end to end.

Milestone 3: Directory

Outcome: the company's members, departments and locations can be managed and browsed as trees; inventory can look them up through contracts.

Database

- ☐ Migration for `directory.departments` (name, code, parent ID, head member ID).
- ☐ Migration for `directory.locations` (name, kind: site/building/floor/room, parent ID).
- ☐ Migration for `directory.members` (employee number, name, email, phone, job title, department, location, manager, status, offboarded at, version) with partial unique indexes on employee number and email for active members.
- ☐ sqlc queries, including recursive CTEs for department and location trees with a depth limit.

Domain and service

- ☐ `Member` with `Offboard`, manager rules (active, not self), and errors.
- ☐ `Department` and `Location` with cycle checks on move; `Location.CanHaveParent` for kind order.
- ☐ Member use cases: `CreateMember`, `UpdateMember`, `OffboardMember`, `GetMember`, `ListMembers` (search, department, location, status filters).
- ☐ Department and location use cases: create, rename, move, list as tree.
- ☐ Events for every change (`directory.member_created`, `directory.member_offboarded`, ...), so they appear in activity.
- ☐ Member import/export from Excel/CSV, reusing the spreadsheet helper built for inventory (can move to Milestone 5).
- ☐ Permission constants and grants in the seeded roles migration.

Contract

- ☐ `contract`: `MemberReader`, `LocationReader`, DTOs, `ErrMemberNotFound`, `ErrLocationNotFound`.

- ☐ `contract/events.go` : public event types and payloads.
- ☐ Contract test checking DTO mapping and not-found errors.

HTTP

- ☐ `openapi.yaml` : `/members` , `/members/{memberID}` , `/members/{memberID}/offboard` , `/departments` , `/departments/{departmentID}` , `/locations` , `/locations/{locationID}` , tree endpoints.
- ☐ Implement the generated interface; generate the TypeScript client.

Tests

- ☐ Domain tests for cycles, location kinds, manager rules.
- ☐ Repository tests for trees and unique constraints mapped to `Conflict` .
- ☐ Handler tests for the main flows and permission checks.

Milestone 4: Inventory core

Outcome: assets can be created with custom fields, tagged, printed as QR labels, nested under parents, checked out to members and checked back in, with full history.

Database

- ☐ Migration for `inventory.statuses` (name, kind, colour, sort order) and settings for default `available` and `in_use` statuses; seed Ordered, In stock, Deployed, In repair, Lost/stolen, Retired, Disposed.
- ☐ Migration for `inventory.categories` (name, parent ID, tag prefix, field definitions as JSONB).
- ☐ Migration for `inventory.assets` (tag unique, name, serial, make, model, category, status, location, parent asset, holder member, checked out at, expected return, purchase date, cost minor units + currency, vendor, order number, warranty expiry, custom fields JSONB, version) with indexes on tag, serial, status, location, holder, and a GIN index on custom fields.
- ☐ Postgres sequence for asset tags; tag generated in `InsertAsset` .
- ☐ sqlc queries: CRUD, filtered list with `sqlc.narg` , count, get for update, asset tree and descendant check via recursive CTE, assets by holder.

Domain

- ☐ `Asset` with `CheckOut`, `CheckIn`, `ChangeStatus`, `AttachTo`, `UpdateDetails`, each recording a domain event.
- ☐ `Category` with `FieldDefinition` and `ValidateCustomFields` (text, number, date, boolean, select; required).
- ☐ `Status` with fixed kinds and the rules that depend on them.
- ☐ Errors: not found, not available, already checked out, cyclic parent, invalid custom field, changed by someone else.
- ☐ Permission constants (`inventory.asset.read`, `.create`, `.update`, `.checkout`, `.import`, `.export`, `inventory.category.manage`, `inventory.status.manage`) and role grants.

Service

- ☐ Asset use cases: `CreateAsset`, `UpdateAsset`, `GetAsset`, `ListAssets`, `AssetTree`, `ChangeStatus`, `AttachToParent`, `DetachFromParent`, `MoveAsset`.
- ☐ Custody use cases: `CheckOut` (validates member via directory's `MemberReader`), `CheckIn`, `AssetsByMember`.
- ☐ Category and status management use cases.
- ☐ Repository save methods map domain events to `contract/events.go` types and call `Outbox.Append` in the same transaction.
- ☐ Optimistic locking: version mismatch returns `ErrAssetChanged` (409).

Contract

- ☐ `contract`: `AssetsByMember` with `AssetSummary` DTO for directory's offboarding page.
- ☐ `contract/events.go`: `AssetCreated`, `AssetUpdated` (changed fields), `AssetStatusChanged`, `AssetCheckedOut`, `AssetCheckedIn`, `AssetMoved`, `AssetAttached`.

HTTP

- ☐ `openapi.yaml`: `/assets` (list with filters, create), `/assets/{assetID}` (get, update), `/assets/{assetID}/check-out`, `/check-in`, `/status`, `/parent`, `/children`, `/assets/{assetID}/qr.svg`, `/members/{memberID}/assets`, `/categories`, `/statuses`.
- ☐ Plain `GET /a/{tag}` redirect to the asset page for QR scans.

- ☐ QR code generation (SVG/PNG) for a single asset and a printable label sheet (PDF) for a selection.
- ☐ Implement the generated interface; generate the TypeScript client.

Tests

- ☐ Domain tests for every custody and status rule, custom field validation and cycles.
- ☐ Service tests with a fake `MemberReader` (inactive member, unknown member).
- ☐ Repository tests: tag generation, filters, optimistic locking, event and job written in the same transaction.
- ☐ End to end: create asset → check out → check in → history shows all three.

Milestone 5: Inventory import and export

Outcome: a new customer can download a template, fill in 800 laptops in Excel, preview errors, fix them, commit, and export the result back to Excel.

Spreadsheet helper

- ☐ `spreadsheet` package: row reader for `.xlsx` (excelize row iterator) and `.csv`, with header detection and blank-row skipping.
- ☐ Row writer using excelize's stream writer, for exports and error reports.
- ☐ Template builder: headers for standard and custom fields, dropdown validation for category and status.
- ☐ Fixture tests: bad headers, empty rows, wrong types, dates as text and as Excel serial numbers, UTF-8 CSV with BOM.

Import

- ☐ Migration for `inventory.imports` (file key, mode, state, counts, created by) and `inventory.import_rows` (row number, action, errors JSON, applied).
- ☐ `Import` entity with states `validated` → `committing` → `completed` / `failed`.
- ☐ `UploadImport`: store the file in `storage`, enforce 10 MB and 10,000 rows, map headers (custom fields by label), preload lookups, validate each row with domain rules, save row results.
- ☐ Matching in `upsert` mode: by tag, then by serial number.

- ☐ `CommitImport` : apply valid rows in batches of 500, mark rows applied, append events; safe to re-run.
- ☐ `worker/commit_import.go` : commits over 1,000 rows run as a River job enqueued with `InsertTx`.
- ☐ Error report: uploaded file plus an error column, served as `.xlsx`.
- ☐ `openapi.yaml` : `POST /imports` (multipart), `GET /imports/{importID}`, `POST /imports/{importID}/commit`, `GET /imports/{importID}/errors.xlsx`, `GET /assets/import-template.xlsx`.

Export

- ☐ `ExportAssets` with the same filters as the asset list and the same columns as import, so a round trip works.
- ☐ `GET /assets/export.xlsx` and `GET /assets/export.csv` streaming up to 5,000 rows; larger exports as a job that stores the file and returns a download link.

Tests

- ☐ Round trip: export → edit in fixture → re-import in upsert mode → assets updated, history entries created.
- ☐ 10,000-row import completes within an agreed time on CI hardware (measure and record the number).

Definition of done

A task is done only when all of these hold:

1. The OpenAPI spec, sqlc queries and migrations are committed, and `make generate` produces no changes.
2. Code follows the layer and contract rules; `make lint` passes, including depguard boundaries.
3. Tests exist at the right layer (domain, service, repository or handler) and `make test` passes.
4. Every state change appends an event, and the change shows up in activity history.
5. Permission checks are in the service, and the seeded roles grant the new permissions.
6. The frontend's generated TypeScript client is regenerated if the spec changed.
7. The guidelines doc is updated if the change introduces a new pattern.